

Distributed Computing Exp-1

9913 Mark Lopes

Be Comps A

Client.c

```
#include <stdio.h>

#include <stdlib.h>

#include <string.h>

#include <winsock2.h>

#include <ws2tcpip.h>

#include <process.h>


#pragma comment(lib, "ws2_32.lib")


#define SERVER_IP "127.0.0.1"

#define PORT 5555

#define BUFFER_SIZE 1024


unsigned __stdcall receive_messages(void *arg) {

    SOCKET client_socket = (SOCKET)arg;

    char buffer[BUFFER_SIZE];

    int rcv_len;


    while ((rcv_len = recv(client_socket, buffer, BUFFER_SIZE, 0)) > 0) {

        buffer[rcv_len] = '\0';

        if (strcmp(buffer, "NICK") == 0) {

            // Server is asking for nickname, main thread will handle this

        } else {

            printf("%s\n", buffer);

        }

    }

}
```

```

if (recv_len <= 0) {
    printf("Disconnected from server.\n");
    closesocket(client_socket);
    exit(0);
}

return 0;
}

int main() {
    WSADATA wsa;
    SOCKET client_socket;
    struct sockaddr_in server_addr;
    char nickname[50];
    char message[BUFFER_SIZE];

    printf("Choose your nickname: ");
    fgets(nickname, 50, stdin);
    nickname[strcspn(nickname, "\n")] = 0; // Remove newline

    printf("Initialising Winsock...\n");
    if (WSAStartup(MAKEWORD(2, 2), &wsa) != 0) {
        printf("Failed. Error Code : %d", WSAGetLastError());
        return 1;
    }
    printf("Initialised.\n");

    if ((client_socket = socket(AF_INET, SOCK_STREAM, 0)) == INVALID_SOCKET) {
        printf("Could not create socket : %d", WSAGetLastError());
    }

    printf("Socket created.\n");

```

```

server_addr.sin_addr.s_addr = inet_addr(SERVER_IP);
server_addr.sin_family = AF_INET;
server_addr.sin_port = htons(PORT);

if (connect(client_socket, (struct sockaddr *)&server_addr, sizeof(server_addr)) < 0) {
    puts("connect error");
    return 1;
}
puts("Connected");

// Wait for NICK prompt
char server_prompt[5];
int recv_len = recv(client_socket, server_prompt, 4, 0);
if (recv_len > 0) {
    server_prompt[recv_len] = '\0';
    if (strcmp(server_prompt, "NICK") == 0) {
        send(client_socket, nickname, strlen(nickname), 0);
    }
}

_beginthreadex(NULL, 0, &receive_messages, (void*)client_socket, 0, NULL);

while (1) {
    fgets(message, BUFFER_SIZE, stdin);
    message[strcspn(message, "\n")] = 0; // Remove newline
    char full_message[BUFFER_SIZE + 52];
    sprintf(full_message, "%s: %s", nickname, message);
    if (send(client_socket, full_message, strlen(full_message), 0) < 0) {
        puts("Send failed");
        return 1;
    }
}

```

```
}

    closesocket(client_socket);

    WSACleanup();

    return 0;
}
```

Server.c

```
#include <stdio.h>

#include <stdlib.h>

#include <string.h>

#include <winsock2.h>

#include <ws2tcpip.h>

#include <process.h>

#pragma comment(lib, "ws2_32.lib")

#define PORT 5555

#define MAX_CLIENTS 10

#define BUFFER_SIZE 1024

typedef struct {
    SOCKET socket;
    char nickname[50];
} client_t;

client_t *clients[MAX_CLIENTS];

int client_count = 0;

void broadcast(char *message, SOCKET sender_socket) {
    for (int i = 0; i < client_count; i++) {
```

```

        if (clients[i]->socket != sender_socket) {
            send(clients[i]->socket, message, strlen(message), 0);
        }
    }
}

```

```

unsigned __stdcall handle_client(void *arg) {
    SOCKET client_socket = (SOCKET)arg;
    char buffer[BUFFER_SIZE];
    int recv_len;

    // Get nickname
    send(client_socket, "NICK", 4, 0);
    recv_len = recv(client_socket, buffer, BUFFER_SIZE, 0);
    if (recv_len <= 0) {
        closesocket(client_socket);
        return 1;
    }
    buffer[recv_len] = '\0';
}

```

```

int current_client_index = -1;
for(int i=0; i<client_count; ++i){
    if(clients[i]->socket == client_socket){
        strcpy(clients[i]->nickname, buffer);
        current_client_index = i;
        break;
    }
}

```

```

printf("Nickname of client is %s\n", buffer);
char join_msg[128];

```

```
sprintf(join_msg, "%s joined the chat!", buffer);
```

```
broadcast(join_msg, client_socket);
```

```
while ((recv_len = recv(client_socket, buffer, BUFFER_SIZE, 0)) > 0) {
```

```
    buffer[recv_len] = '\0';
```

```
    printf("[MESSAGE] %s\n", buffer);
```

```
    broadcast(buffer, client_socket);
```

```
}
```

```
closesocket(client_socket);
```

```
if(current_client_index != -1){
```

```
    char leave_msg[128];
```

```
    sprintf(leave_msg, "%s left the chat.", clients[current_client_index]->nickname);
```

```
    broadcast(leave_msg, client_socket);
```

```
    free(clients[current_client_index]);
```

```
    for(int i = current_client_index; i < client_count - 1; i++){
```

```
        clients[i] = clients[i+1];
```

```
    }
```

```
    client_count--;
```

```
}
```

```
return 0;
```

```
}
```

```
int main() {
```

```
    WSADATA wsa;
```

```
    SOCKET server_socket, client_socket;
```

```
    struct sockaddr_in server_addr, client_addr;
```

```
    int client_addr_len = sizeof(client_addr);
```

```
    printf("Initialising Winsock...\n");
```

```
    if (WSAStartup(MAKEWORD(2, 2), &wsa) != 0) {
```

```

    printf("Failed. Error Code : %d", WSAGetLastError());
    return 1;
}

printf("Initialised.\n");

if ((server_socket = socket(AF_INET, SOCK_STREAM, 0)) == INVALID_SOCKET) {
    printf("Could not create socket : %d", WSAGetLastError());
}

printf("Socket created.\n");

server_addr.sin_family = AF_INET;
server_addr.sin_addr.s_addr = INADDR_ANY;
server_addr.sin_port = htons(PORT);

if (bind(server_socket, (struct sockaddr *)&server_addr, sizeof(server_addr)) == SOCKET_ERROR) {
    printf("Bind failed with error code : %d", WSAGetLastError());
    exit(EXIT_FAILURE);
}

puts("Bind done");

listen(server_socket, 3);

puts("Waiting for incoming connections...");

while ((client_socket = accept(server_socket, (struct sockaddr *)&client_addr, &client_addr_len))
!= INVALID_SOCKET) {
    puts("Connection accepted");

    if (client_count < MAX_CLIENTS) {
        client_t *new_client = (client_t *)malloc(sizeof(client_t));
        new_client->socket = client_socket;
    }
}

```

```

        clients[client_count++] = new_client;

        _beginthreadex(NULL, 0, &handle_client, (void*)client_socket, 0, NULL);
    } else {

        char *message = "Server is full.";

        send(client_socket, message, strlen(message), 0);

        closesocket(client_socket);

    }
}

if (client_socket == INVALID_SOCKET) {

    printf("accept failed with error code : %d", WSAGetLastError());

    return 1;

}

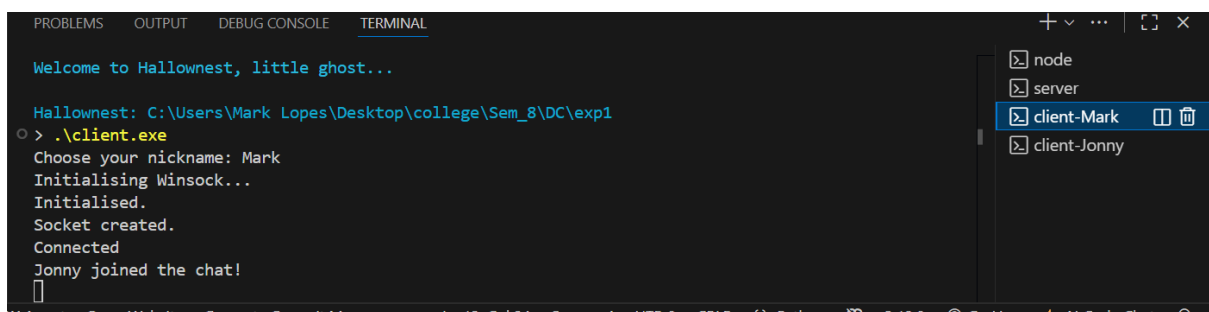
closesocket(server_socket);

WSACleanup();

return 0;

}

```

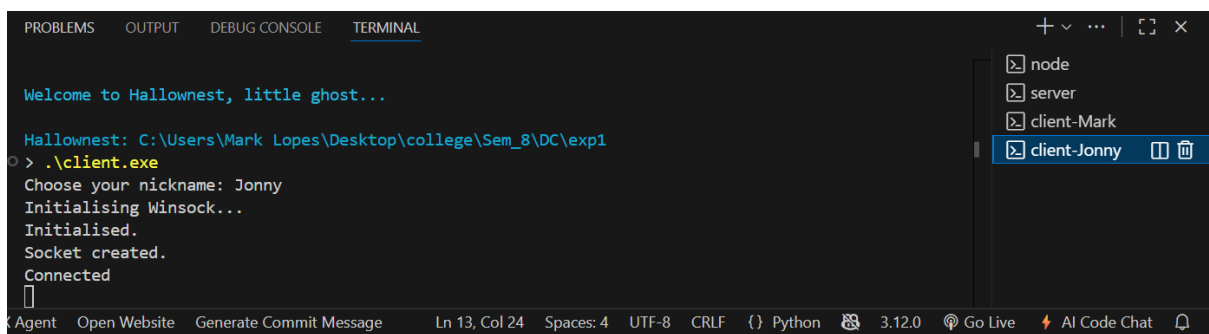


```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL
Welcome to Hallownest, little ghost...

Hallownest: C:\Users\Mark Lopes\Desktop\college\Sem_8\DC\exp1
> .\client.exe
Choose your nickname: Mark
Initialising Winsock...
Initialised.
Socket created.
Connected
Jonny joined the chat!

```



```

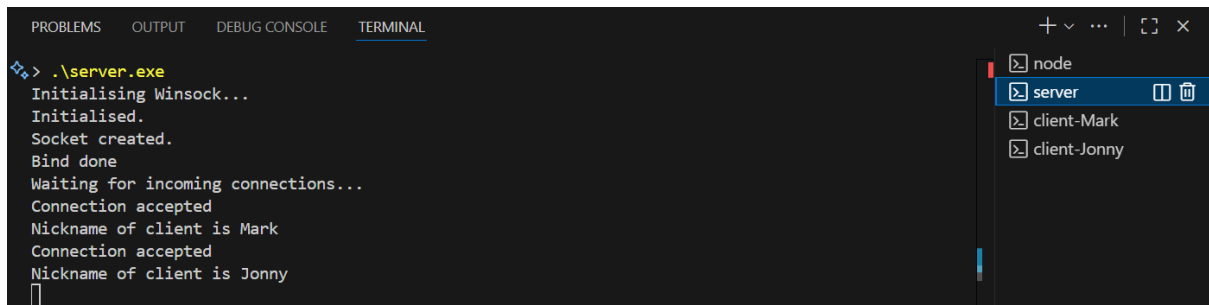
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL
Welcome to Hallownest, little ghost...

Hallownest: C:\Users\Mark Lopes\Desktop\college\Sem_8\DC\exp1
> .\client.exe
Choose your nickname: Jonny
Initialising Winsock...
Initialised.
Socket created.
Connected

```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL
+ v ... | [ ] x
❖ > .\server.exe
Initialising Winsock...
Initialised.
Socket created.
Bind done
Waiting for incoming connections...
Connection accepted
Nickname of client is Mark
Connection accepted
Nickname of client is Jonny

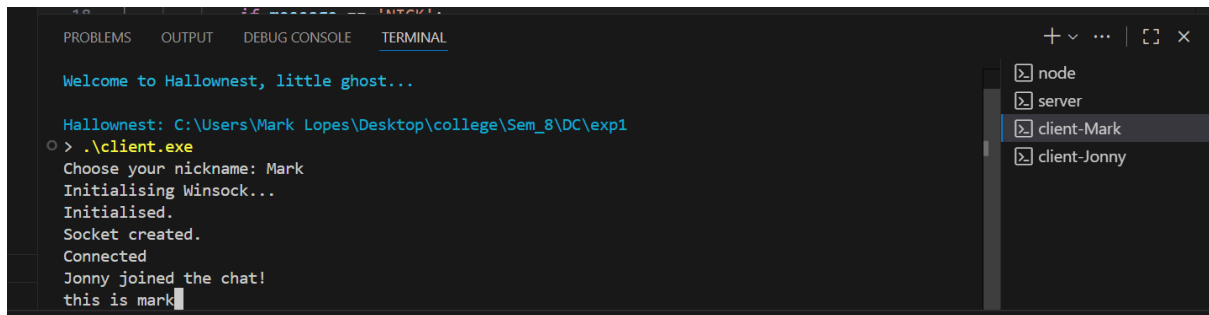
```



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL
+ v ... | [ ] x
Welcome to Hallownest, little ghost...

Hallownest: C:\Users\Mark Lopes\Desktop\college\Sem_8\DC\exp1
> .\client.exe
Choose your nickname: Mark
Initialising Winsock...
Initialised.
Socket created.
Connected
Jonny joined the chat!
this is mark

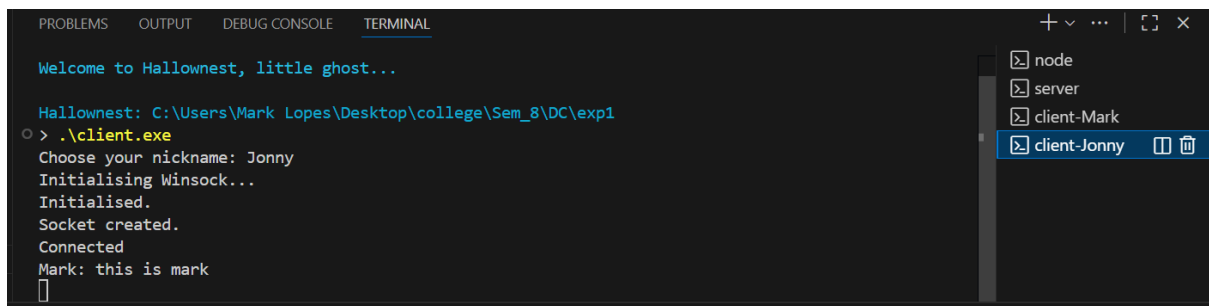
```



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL
+ v ... | [ ] x
Welcome to Hallownest, little ghost...

Hallownest: C:\Users\Mark Lopes\Desktop\college\Sem_8\DC\exp1
> .\client.exe
Choose your nickname: Jonny
Initialising Winsock...
Initialised.
Socket created.
Connected
Mark: this is mark

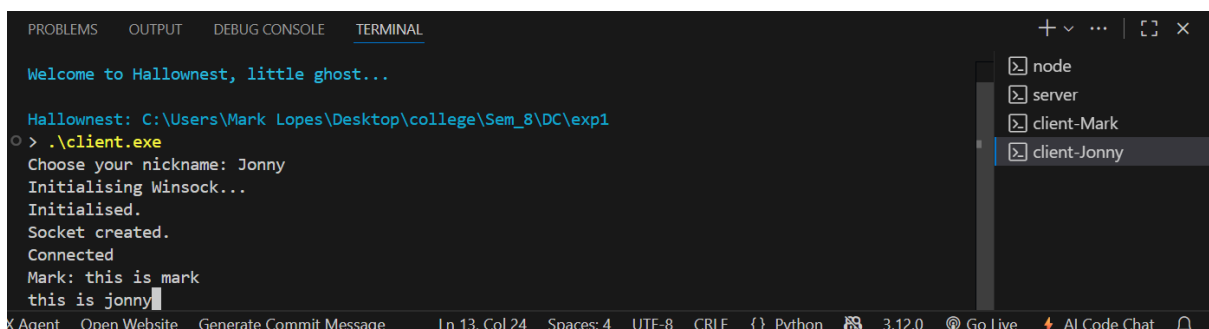
```



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL
+ v ... | [ ] x
Welcome to Hallownest, little ghost...

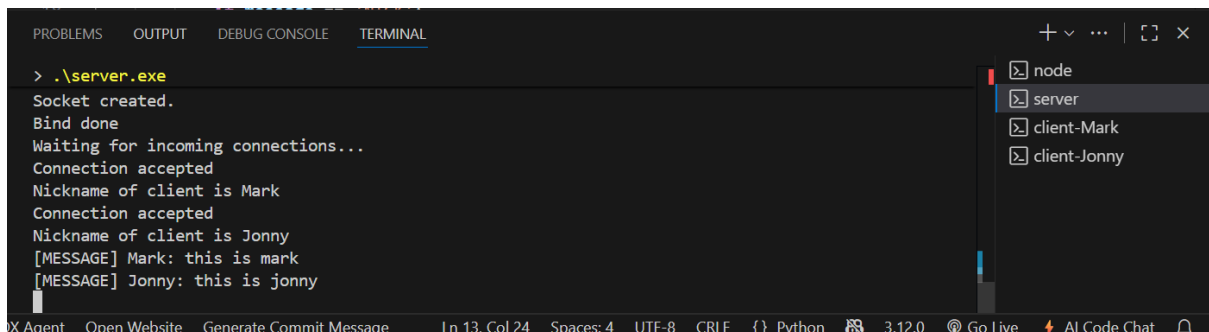
Hallownest: C:\Users\Mark Lopes\Desktop\college\Sem_8\DC\exp1
> .\client.exe
Choose your nickname: Jonny
Initialising Winsock...
Initialised.
Socket created.
Connected
Mark: this is mark
this is jonny

```

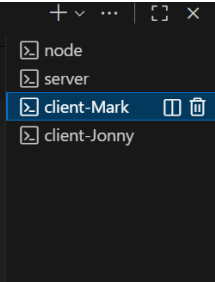


```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL
+ v ... | [ ] x
> .\server.exe
Socket created.
Bind done
Waiting for incoming connections...
Connection accepted
Nickname of client is Mark
Connection accepted
Nickname of client is Jonny
[MESSAGE] Mark: this is mark
[MESSAGE] Jonny: this is jonny

```

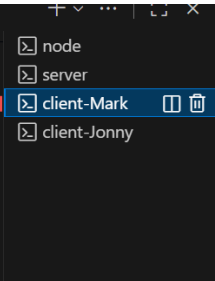


```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL
Hallownest: C:\Users\Mark Lopes\Desktop\college\Sem_8\DC\exp1
> .\client.exe
Choose your nickname: Mark
Initialising Winsock...
Initialised.
Socket created.
Connected
Jonny joined the chat!
this is mark
Jonny: this is jonny
^
```

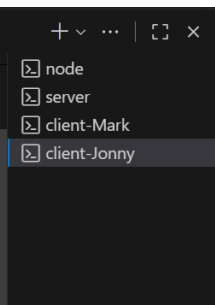


```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL
> .\client.exe
Initialising Winsock...
Initialised.
Socket created.
Connected
Jonny joined the chat!
this is mark
Jonny: this is jonny

Hallownest: C:\Users\Mark Lopes\Desktop\college\Sem_8\DC\exp1
^>
```



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL
> .\client.exe
Choose your nickname: Jonny
Initialising Winsock...
Initialised.
Socket created.
Connected
Mark: this is mark
this is jonny
Mark: this is mark
Mark left the chat.
^V
```



Postlab:-

9913-Mark Lopes
BE Comp A
DC - Exp 1 Postlab

Q.1 Differentiate between a process and a thread.

Process	Thread
1. It is an independent program in execution with its own memory space and system resources.	1. It is a smaller unit of process that share its memory and resources.
2. Each process has a separate memory space.	2. Threads of same process share same memory space.
3. Creating a new process is expensive and slower.	3. Creating a thread is lightweight and faster.
4. A crash in one process does not affect others.	4. A crash in one thread affects the entire process.

In a multithreaded server, each new client connection is handled by a separate thread.

This allows multiple clients to be served concurrently.

Each thread independently manages communication with its respective client.

The main server thread continues listening for new connection instead of being blocked.

Q.2 What is a thread pool? Explain how a multithreaded server with a thread pool processes incoming client requests.

→ A thread pool is a collection of pre-created threads that are kept ready to perform tasks.

Instead of creating a new thread for every client request, the server reuses threads from the pool.

1. The server starts with a fixed number of threads (pool).
2. When client request arrives:-
 - The request is placed in queue.
 - An available thread from the pool picks up the request and processes it.
3. Once task is done, the thread returns to pool and waits for new task.
4. This continues without creating or destroying threads repeatedly.

Q.3 Give some examples of Multithreaded Servers and clients.

→ I] Multithreaded Servers:-

i) Apache http server

Uses threads to handle multiple web clients simultaneously.

ii) Tomcat.

Uses thread pool to handle concurrent http requests efficiently.

iii) chat servers (whatsapp)

Each client connection is handled by a separate thread or from a thread pool.

II] Multithreaded clients:-

i) Web browsers

Use multiple threads to load webpage components.

ii) Download managers

Use multiple threads to download different parts of file simultaneously.

iii) Multiplayer Game Clients

Use separate threads for networking, rendering and user input handling.